

Bitwise Manipulation

371. Sum of Two Integers

To a Google L5 or L6 engineer, this problem isn't about finding a "loophole" around the `+` operator. It is a fundamental test of your understanding of **Digital Logic** and how a CPU actually performs arithmetic at the hardware level using logic gates.

This problem asks you to build a "Half-Adder" and "Full-Adder" circuit using software code.

1. Problem Explanation

The Goal: Calculate the sum of two integers `a` and `b`, but you are **strictly forbidden** from using the `+` or `-` operators.

The Constraint: Since we cannot use standard arithmetic operators, we must drop down into the world of **Binary** and **Bitwise Operations**. We have to mimic how we did addition in grade school, but in base-2 (binary).

Grade School Addition Recap: When you add $15 + 7$: 1. You add the digits: $5 + 7 = 12$. 2. You write down the 2 and **carry** the 1 to the next column. 3. In the next column, you add the carry: $1 + 1 = 2$. 4. Result: 22.

In binary, we do the exact same thing, but we use logic gates to handle the "sum" and the "carry."

2. Solution Explanation

The Intuition (The "Why"): We use two specific bitwise operators to replace the `+` operator:

1. **XOR ($\wedge$) - The "Partial Sum":** In binary, XOR behaves exactly like addition **if you ignore the carry**.
 - $0 \wedge 0 = 0$
 - $0 \wedge 1 = 1$
 - $1 \wedge 0 = 1$
 - $1 \wedge 1 = 0$ (Wait, $1+1$ should be 10, but XOR just gives us the 0. It "forgets" the carry).
2. **AND ($\&$) followed by Shift ($\ll$) - The "Carry":** In binary, a carry only happens when both bits are 1.
 - $1 \& 1 = 1$.
 - We then shift this 1 to the left ($\ll 1$) because a carry always moves to the next power-of-two column.

[Image of half adder circuit logic gates]

The Algorithm (The “How”): 1. Calculate the partial sum using $a \oplus b$. 2. Calculate the carry using $(a \& b) \ll 1$. 3. Now, we need to add the partial sum and the carry. Since we can't use $+$, we repeat the process: treat the sum as the new a and the carry as the new b . 4. Keep going until the carry becomes 0.

Detailed ASCII Walkthrough: Let's add $a = 5$ and $b = 3$.

CONVERSION TO BINARY:

5 is 0101

3 is 0011

ROUND 1:

a: 0 1 0 1 (5)

b: 0 0 1 1 (3)

Step 1: Get Partial Sum ($a \oplus b$)

 0 1 0 1
 $\oplus$ 0 0 1 1

 0 1 1 0 => (This is 6)

Step 2: Get Carry ($a \& b \ll 1$)

 0 1 0 1
 $\&$ 0 0 1 1

 0 0 0 1 => (Carry at the 1s place)

Shifted: 0 0 1 0 => (This is 2)

Current State: Sum = 6, Carry = 2. We must add them.

ROUND 2:

a: 0 1 1 0 (6)

b: 0 0 1 0 (2)

Step 1: Partial Sum ($a \oplus b$)

 0 1 1 0
 $\oplus$ 0 0 1 0

 0 1 0 0 => (This is 4)

Step 2: Carry ($a \& b \ll 1$)

 0 1 1 0
 $\&$ 0 0 1 0

```

-----
  0 0 1 0 => (Carry at the 2s place)
  Shifted: 0 1 0 0 => (This is 4)

Current State: Sum = 4, Carry = 4. We must add them.
-----

ROUND 3:
a: 0 1 0 0 (4)
b: 0 1 0 0 (4)

Step 1: Partial Sum (a ^ b)
  0 1 0 0
^  0 1 0 0
-----
  0 0 0 0 => (This is 0)

Step 2: Carry (a & b) << 1
  0 1 0 0
&  0 1 0 0
-----
  0 1 0 0 => (Carry at the 4s place)
  Shifted: 1 0 0 0 => (This is 8)

Current State: Sum = 0, Carry = 8.
-----

ROUND 4:
a: 0 0 0 0 (0)
b: 1 0 0 0 (8)

Step 1: Partial Sum (0 ^ 8) = 1 0 0 0 (8)
Step 2: Carry (0 & 8) << 1 = 0 0 0 0 (0)

Carry is 0. We STOP.
FINAL RESULT: 8

```

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(1)$ * -----
 While it looks like $O(N)$, integers in most languages are fixed-size (32-bit or 64-bit).

- The maximum number of rounds we can go is equal to the number of bits in the integer.
- For a 32-bit integer, we will loop at most 32 times.

- Since 32 is a constant, the time is $O(1)$.

```
[ TC Derivation ]
Input A (32 bits)
Input B (32 bits)
Loop: 1 -> 2 -> ... -> 32 (max)
Operations: Constant bound.
```

SPACE COMPLEXITY: $O(1)$

- We only use two variables to store the sum and the carry.
- We do not use any arrays, recursion stacks, or additional memory structures.

```
[ Memory Map ]
[ Variable A: 4 bytes ]
[ Variable B: 4 bytes ]
Total extra space: 8 bytes (Constant).
```

4. Solution Code

Special Note on Python: Python integers are “arbitrary precision” (they can grow forever). They don’t naturally overflow at 32 bits like C++ or Java. To handle negative numbers and 32-bit limits, we use a **mask** 0xFFFFFFFF.

Python Snippet

```
def getSum(a: str, b: str) -> int:
    # 32 bits integer max
    MAX = 0x7FFFFFFF
    # Mask to keep only 32 bits
    MASK = 0xFFFFFFFF

    while b != 0:
        # a ^ b computes the sum without carries
        # (a & b) << 1 computes the carries
        # We apply the MASK to keep the numbers within 32-bit range
        a, b = (a ^ b) & MASK, ((a & b) << 1) & MASK

    # If a is negative (greater than MAX), we need to
    # format it correctly for Python's long integers
    return a if a <= MAX else ~(a ^ MASK)
```

JavaScript Snippet

```
/**
 * JavaScript handles bitwise operations on 32-bit signed integers
 * by default, making this much simpler than Python.
 */
function getSum(a, b) {
  // Keep looping as long as there is a carry to add
  while (b !== 0) {
    // Step 1: Calculate sum without carry
    let partialSum = a ^ b;

    // Step 2: Calculate carry and shift it to the next position
    // The bitwise & finds where both are 1.
    // The << 1 moves those carries to the next power of 2.
    let carry = (a & b) << 1;

    // Update a and b for the next round
    a = partialSum;
    b = carry;
  }

  return a;
}
```

Note 1: Terminology Addendum

Bitwise XOR (^): * **Logic:** Returns 1 if the bits are different, 0 if they are the same. * **Why it helps:** It mimics binary addition for (0+0), (0+1), and (1+0). It only fails at (1+1), which it treats as 0.

Bitwise AND (&): * **Logic:** Returns 1 only if both bits are 1. * **Why it helps:** It identifies the exact columns where a “carry” is generated.

Two’s Complement: * **What it is:** The standard way computers represent negative numbers in binary. * **How it applies:** This bitwise solution works for both positive and negative numbers automatically because of how Two’s Complement arithmetic is designed.

Note 2: Real-World Interview Variations

Variation 1: “Subtract Two Integers” (Bloomberg/Meta) * **Prompt:** Subtract *b* from *a* without using *-*. * **The L5 Solve:** Senior engineers know that $a - b$ is the same as $a + (\sim b + 1)$. You flip all bits of *b* (NOT operator), add 1 using your *getSum* function, and then add that result to *a*.

Variation 2: “Number of 1 Bits” / “Hamming Weight” (Google) *
Prompt: Count how many set bits (1s) are in an integer. * **The L5 Solve:** Use **Brian Kernighan’s Algorithm**. In a loop, perform $n = n \& (n - 1)$. This operation always clears the least-significant set bit. Count how many times you can do this before n becomes 0.

Variation 3: “Single Number” (Meta) * **Prompt:** Every element in an array appears twice except for one. Find it without extra space. * **The L5 Solve:** XOR all numbers together. Since $x \oplus x = 0$, all pairs will cancel out, leaving only the single number. This demonstrates your ability to use bitwise properties for space optimization.

191. Number of 1 Bits

This is a classic problem that tests a candidate’s knowledge of **Bit Manipulation**. While a junior engineer might solve this by checking every single bit, a Google L5/L6 engineer will use a specific trick called **Brian Kernighan’s Algorithm** to skip the “zeros” and only look at the “ones.”

1. Problem Explanation

The Premise: You are given a positive integer n . You need to count how many “set bits” (the number 1) it has in its binary representation. This is also known as the **Hamming Weight**.

The Intuition (The “Why”): Computers store numbers as a series of 0s and 1s. For example, the number 11 is stored as 1011. To solve this, we need to “peek” into the binary structure.

Standard approach: Check every bit (there are 32 bits in a standard integer). If the bit is 1, increment a counter. **Senior approach:** Use a mathematical property to “snuff out” the rightmost 1-bit in every step. If a number has only three 1-bits, we should only have to do three operations, even if the number is huge.

2. Solution Explanation

We will focus on **Brian Kernighan’s Algorithm**.

The Core Trick: $n = n \& (n - 1)$ When you subtract 1 from a binary number, the rightmost 1 becomes a 0, and all the 0s to its right become 1s. If you then perform a Bitwise AND ($\&$) between the original number and the subtracted number, that rightmost 1 is effectively erased.

Detailed ASCII Walkthrough Let's trace $n = 12$. In binary, 12 is 1100.

Step 1: Initial State $n = 12$ (Binary: 1100) $\text{count} = 0$

Step 2: First Operation We subtract 1: $12 - 1 = 11$ (Binary: 1011) Perform AND: 1100 & 1011

```
  1 1 0 0  (n = 12)
& 1 0 1 1  (n - 1 = 11)
-----
  1 0 0 0  (Result = 8)
```

n is now 8. count becomes 1. *Notice: The rightmost 1 at the "4s" position is gone.*

Step 3: Second Operation $n = 8$ (Binary: 1000) We subtract 1: $8 - 1 = 7$ (Binary: 0111) Perform AND: 1000 & 0111

```
  1 0 0 0  (n = 8)
& 0 1 1 1  (n - 1 = 7)
-----
  0 0 0 0  (Result = 0)
```

n is now 0. count becomes 2. *Notice: The last 1 is gone.*

Step 4: Termination Since n is now 0, we stop. Final Count: 2.

3. Time and Space Complexity Analysis

Time Complexity: $O(1)$ or $O(S)$ Where S is the number of set bits (1s).

Visual Derivation of Time Complexity:

Scenario: n is a 32-bit integer.

Approach A (Junior): Checks every bit.

[1][0][1][0][...] $\rightarrow$ 32 total checks.

Complexity: $O(32)$ which is Constant $O(1)$.

Approach B (Brian Kernighan): Only checks 1s.

If $n = 1000\dots000$ (only one bit set)

Step 1: [0][0][0][...] $\rightarrow$ Done.

Complexity: $O(\text{Number of set bits})$.

Since the number of bits is capped at 32 (or 64), the time is technically Constant $O(1)$.

Space Complexity: $O(1)$

Visual Derivation of Space Complexity:

Variables used:

[count] -> Integer
[n] -> Integer (modified in place)

No arrays, sets, or recursive stacks are used.

Memory used does not grow with the size of the number.

Space: $O(1)$.

4. Solution Code

Python Implementation

```
def hammingWeight(n: int) -> int:
    count = 0
    # This loop runs exactly as many times as there are 1-bits
    while n != 0:
        # Brian Kernighan's bit-flipping trick
        # It clears the least significant (rightmost) set bit
        n = n & (n - 1)
        count += 1
    return count
```

JavaScript Implementation

```
/**
 * Uses Brian Kernighan's Algorithm to count set bits.
 * This is more efficient than checking all 32 bits individually.
 */
var hammingWeight = function(n) {
    let count = 0;

    // In JS, bitwise operations treat numbers as 32-bit signed integers.
    // We use a while loop that terminates when no set bits remain.
    while (n !== 0) {
        // The AND operation with (n-1) erases the rightmost 1.
        n = n & (n - 1);
        count++;
    }

    return count;
};
```

Note 1: Terms and Techniques

- **Bitwise AND (&):** A logic gate that outputs 1 only if both input bits are 1. It is the fundamental building block of bit manipulation.
 - **Brian Kernighan’s Algorithm:** A famous optimization for counting set bits. It is highly valued in systems programming (like OS kernels or embedded systems) because it avoids unnecessary loop iterations.
 - **Set Bit:** A bit that has a value of 1. A “cleared” bit has a value of 0.
-

Note 2: Real-World Interview Variations

1. Google: “Hamming Distance” (LeetCode 461) * The Twist: Given two numbers x and y , find the number of positions where the bits are different. * **L5 Solution:** First, perform a Bitwise XOR ($x \oplus y$). The XOR result will have a 1 only in positions where the bits were different. Then, use the `hammingWeight` code above to count the 1s in that XOR result.

2. Meta: “Power of Two” (LeetCode 231) * The Twist: Determine if a given integer is a power of two. * **L5 Solution:** Use the same trick! A power of two (like 2, 4, 8, 16) has exactly **one** set bit in binary (e.g., 8 is 1000). If $(n \& (n - 1)) == 0$ and $n > 0$, it is a power of two. This is an $O(1)$ solution.

3. Bloomberg: “IP Address Masking” * The Twist: You are given a subnet mask (like 255.255.255.0). You need to count the “prefix length” (how many 1s are in the mask). * **L5 Solution:** Convert the IP segments into their 32-bit integer equivalent and apply the `hammingWeight` algorithm. This is used in network engineering to determine how many bits are allocated for the network vs. the host.

338. Counting Bits

A Google L5 or L6 engineer sees this problem and immediately recognizes that “Counting Bits” for a range of numbers is not a bit-manipulation problem—it is a **Dynamic Programming** problem.

The goal for a senior engineer is to avoid calculating the bits for each number from scratch. Instead, they look for a relationship where the answer for a large number can be derived from the answer of a smaller number we have already computed.

1. Problem Explanation

The Goal: Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the **number of 1’s** in the binary representation of i .

The Bruteforce Way (What to avoid): For every number from 0 to n , run a loop to count its 1s. If n is 1 million, and each number has up to 32 bits, you are doing 32 million operations. A senior engineer wants to do this in exactly 1 million operations.

The Intuition: Let's look at the binary patterns and see if we can find a "recyclable" pattern.

Decimal	Binary	Number of 1s
0	0000	0
1	0001	1
2	0010	1
3	0011	2
4	0100	1
5	0101	2
6	0110	2
7	0111	3

2. Solution Explanation

To solve this in $O(n)$ time, we use a pattern called **Bit Shifting + Parity**.

The Observation: Look at the relationship between a number and that same number shifted right by 1 (which is the same as $i / 2$ in integer math).

- **Case 1: Even Numbers** Take the number **6** (Binary 110). If we shift it right ($6 \gg 1$), we get **3** (Binary 011). Notice that the number of 1s is exactly the same! This is because an even number always ends in 0. Shifting right just removes that 0.
- **Case 2: Odd Numbers** Take the number **7** (Binary 111). If we shift it right ($7 \gg 1$), we get **3** (Binary 011). The number of 1s in **7** is the number of 1s in **3** plus **1** (for the 1 we chopped off at the end).

The DP Relation (The "Why"): Number of 1s in $i = (\text{Number of 1s in } i \gg 1) + (1 \text{ if } i \text{ is odd, else } 0)$.

ASCII Diagram Walkthrough: $n = 5$ We initialize an array `dp = [0, 0, 0, 0, 0]`.

Step 1: $i = 1$

```

Binary: 001
i >> 1 = 0
dp[1] = dp[0] + (1 % 2)
       = 0 + 1 = 1
Current DP: [0, 1, 0, 0, 0]
```

Step 2: i = 2

```

Binary: 010
i >> 1 = 1
dp[2] = dp[1] + (2 % 2)
      = 1 + 0 = 1
Current DP: [0, 1, 1, 0, 0, 0]

```

Step 3: i = 3

```

Binary: 011
i >> 1 = 1
dp[3] = dp[1] + (3 % 2)
      = 1 + 1 = 2
Current DP: [0, 1, 1, 2, 0, 0]

```

Step 4: i = 4

```

Binary: 100
i >> 1 = 2
dp[4] = dp[2] + (4 % 2)
      = 1 + 0 = 1
Current DP: [0, 1, 1, 2, 1, 0]

```

Step 5: i = 5

```

Binary: 101
i >> 1 = 2
dp[5] = dp[2] + (5 % 2)
      = 1 + 1 = 2
Final DP: [0, 1, 1, 2, 1, 2]

```

3. Time and Space Complexity Analysis

```

+-----+
| TIME COMPLEXITY DERIVATION: O(n) |
+-----+
| 1. We initialize an array of size n+1. |
| 2. We iterate from 1 to n exactly one time. |
| 3. Inside the loop, we perform two O(1) operations: |
|   - A bit shift (or division by 2). |
|   - A modulo (or bitwise AND) to check parity. |
| |
| TOTAL TIME: O(n) operations. |
+-----+

+-----+
| SPACE COMPLEXITY DERIVATION: O(1) or O(n) |
+-----+

```

```

+-----+
| 1. We create an array 'ans' to store the results. |
| 2. Per LeetCode standards, the space used for the output array |
|    is usually not counted as auxiliary space. |
| 3. We use zero extra data structures (no stacks, heaps, or maps). |
| |
| TOTAL AUXILIARY SPACE: O(1) |
+-----+

```

4. Solution Code

Optimized DP Solution (The L5 Standard) Python Snippet

```

def countBits(n: int) -> list[int]:
    # Initialize DP table with 0s
    dp = [0] * (n + 1)

    # Iterate from 1 to n. 0 is already correctly set to 0.
    for i in range(1, n + 1):
        # i >> 1 is i divided by 2 (removes the last bit)
        # i & 1 is i modulo 2 (checks if the last bit is a 1)
        # We reuse the previously calculated value for i >> 1
        dp[i] = dp[i >> 1] + (i & 1)

    return dp

```

JavaScript Snippet

```

/**
 * @param {number} n
 * @return {number[]}
 */
var countBits = function(n) {
    // Create an array of size n + 1, filled with 0
    const dp = new Array(n + 1).fill(0);

    for (let i = 1; i <= n; i++) {
        // dp[i >> 1] gets the number of 1s in the number without its last bit
        // (i & 1) returns 1 if the last bit is 1, and 0 otherwise
        dp[i] = dp[i >> 1] + (i & 1);
    }

    return dp;
};

```

Note 1: Terminology and Techniques

- **DP (Dynamic Programming):** A technique where you solve a complex problem by breaking it down into simpler subproblems and storing the results of those subproblems so you don't have to recompute them.
 - **Bitwise Right Shift (>>):** Shifting a binary number to the right by one position. It effectively divides the number by 2 and discards the remainder.
 - **Bitwise AND (&):** A logic operation. Doing $i \& 1$ acts as a mask that only looks at the very last bit of a number. If the bit is 1, the result is 1. If the bit is 0, the result is 0.
-

Note 2: Real-World Interview Variations

1. Google: “Efficient Set-Bit Tracking” * **Scenario:** You are building a high-speed networking tool that needs to calculate the “Hamming Weight” (number of set bits) for millions of packet IDs per second. * **Solution:** A senior engineer would suggest **Precomputing a Lookup Table**. Instead of calculating bits on the fly, you precalculate the bits for all numbers from 0 to 255 (1 byte) using the DP method above. Then, for any 32-bit integer, you just break it into 4 bytes and sum their precomputed values from the table. This is even faster than $O(n)$.

2. Meta: “Hamming Distance in Social Graphs” * **Scenario:** Given two user IDs (as integers), find the number of positions at which the corresponding bits are different (Hamming Distance). * **Solution:** XOR the two numbers together ($\text{xor_result} = a \oplus b$). This creates a new number where a 1 exists only where the bits were different. Then, use the DP logic or a built-in bit-counting function to count the 1s in `xor_result`.

3. Bloomberg: “Stock Ticker Permissions” * **Scenario:** Access levels for various financial feeds are stored as bitmasks in an integer. Write a function to check if a user has exactly k permissions enabled. * **Solution:** This requires counting set bits. If this needs to be checked frequently for a range of users, implementing the $O(n)$ DP approach ensures the permission engine remains highly performant during peak trading hours.

268. Missing Number

At the L5/L6 level at Google, we view “Missing Number” (LeetCode 268) as a test of **efficiency and mathematical intuition**. While a junior engineer might suggest sorting the array, a senior engineer looks for a “one-pass” solution that uses constant extra memory.

There are three ways to solve this. I will focus on the two most elegant: **Gauss’s Summation** (Mathematical) and **Bit Manipulation (XOR)**.

1. Problem Explanation

The Core Question: You are given an array containing n distinct numbers in the range $[0, n]$. This means the array has a length of n , but it is supposed to cover $n + 1$ possible values. One value is missing. Find it.

The Constraints: * All numbers are unique. * The numbers are between 0 and n . * You must find the missing one in linear time.

Visualizing the Gap: Input: `nums = [3, 0, 1]`, $n = 3$ The numbers should be: 0, 1, 2, 3

EXPECTED: [0, 1, 2, 3] (Sum = 6)

ACTUAL: [3, 0, 1] (Sum = 4)

|

v

Missing = Expected - Actual = 6 - 4 = 2

2. Solution Explanation

Approach 1: Gauss's Summation (The "Balance Scale" Method) If we know what the total sum *should* be for a range of 0 to n , and we calculate what the sum *actually* is, the difference between the two must be the missing number.

The Formula: The sum of numbers from 0 to n is $(n * (n + 1)) / 2$.

Walkthrough: `nums = [0, 1, 3]`, $n = 3$

Step 1: Calculate Expected Sum

$n = 3$

Expected = $(3 * (3 + 1)) / 2$

Expected = $(3 * 4) / 2 = 6$

Step 2: Calculate Actual Sum

Actual = $0 + 1 + 3 = 4$

Step 3: Subtract

Result = $6 - 4 = 2$

Approach 2: Bit Manipulation (The "Mirror" Method) This uses the XOR operator. XOR has a magical property: $A \oplus A = 0$ and $A \oplus 0 = A$. If we XOR every index from 0 to n with every value in the array, every number that appears in both sets will cancel itself out, leaving only the missing number behind.

Walkthrough Visualization: nums = [3, 0, 1], n = 3

Indices (Expected): 0, 1, 2, 3

Values (Actual): 3, 0, 1

Let's XOR them all together:

$(0 \wedge 3) \wedge (1 \wedge 0) \wedge (2 \wedge 1) \wedge 3$

Rearrange (Order doesn't matter for XOR):

$(0 \wedge 0) \wedge (1 \wedge 1) \wedge (3 \wedge 3) \wedge 2$

v		v		v		v
0	$\wedge$	0	$\wedge$	0	$\wedge$	2 = 2

3. Time and Space Complexity Analysis

Let n be the length of the input array.

=====

APPROACH: GAUSS SUMMATION or XOR

=====

TIME COMPLEXITY: $O(n)$

Derivation Diagram:

[Input Array] (Length n)

--> (Gauss) We loop once to sum the elements.
--> (XOR) We loop once to XOR the elements.
v

Total operations scale directly with n.

Total Time = $O(n)$

=====

SPACE COMPLEXITY: $O(1)$

=====

Derivation Diagram:

[Memory Used]

--> (Gauss) We store two integers: 'expected' and 'actual'.
--> (XOR) We store one integer: 'res'.
v

No matter how large the array gets, we only use a fixed number of variables. We do not create any new lists or tables.

Total Space = $O(1)$

=====

4. Solution Code

JavaScript

```
/**
 * Approach 1: Gauss's Summation
 * Optimized for clarity and speed.
 */
function missingNumberSum(nums) {
    const n = nums.length;
    // Calculate expected sum of 0...n
    const expectedSum = (n * (n + 1)) / 2;

    // Calculate actual sum of array elements
    let actualSum = 0;
    for (let num of nums) {
        actualSum += num;
    }

    return expectedSum - actualSum;
}

/**
 * Approach 2: XOR Bit Manipulation
 * Highly efficient, prevents integer overflow for massive n.
 */
function missingNumberXOR(nums) {
    let res = nums.length;

    // We XOR the index and the value at the same time
    for (let i = 0; i < nums.length; i++) {
        res = res ^ i ^ nums[i];
    }

    return res;
}
```

Python

```
# Approach 1: Gauss's Summation
# Time:  $O(n)$ , Space:  $O(1)$ 
def missingNumberSum(nums: list[int]) -> int:
```

```

n = len(nums)
# The sum of 0...n is (n * (n + 1)) / 2
expected_sum = n * (n + 1) // 2
actual_sum = sum(nums)

return expected_sum - actual_sum

# Approach 2: XOR Bit Manipulation
# Time: O(n), Space: O(1)
def missingNumberXOR(nums: list[int]) -> int:
    # Initialize with the length n because the loop only goes to n-1
    res = len(nums)

    # Process each index and its value
    for i, num in enumerate(nums):
        # res = (original_n ^ index_0 ^ val_0 ^ index_1 ^ val_1 ...)
        res ^= i ^ num

    return res

```

Note 1: Terms and Techniques

- **Gauss Summation:** Named after Carl Friedrich Gauss. It's a way to find the sum of an arithmetic progression. It helps here by providing a "gold standard" to compare our array against.
 - **XOR (Exclusive OR):** A bitwise operator. It is useful for finding unique or missing elements because it "cancels out" duplicate information. It is often safer than summation for very large numbers where a sum might exceed the maximum size of an integer (integer overflow).
-

Note 2: Real-World & Interview Variations

1. Google: "Find the Duplicate Number" (LeetCode 287) * The Problem: One number is repeated instead of one missing. *** The Solution:** You can't use sum or XOR as easily if the range is different. Instead, you treat the array as a **Linked List** (where `nums[i]` is a pointer to the next index) and use **Floyd's Cycle-Finding Algorithm** (Slow/Fast pointers) to find the entrance to the cycle.

2. Meta: "Single Number" (LeetCode 136) * The Problem: Every element appears twice except for one. Find it. *** The Solution:** This is the pure application of the **XOR** technique. XORing all numbers in the array results in the single number, as all pairs cancel out.

3. Bloomberg: “Find All Numbers Disappeared in an Array” (LeetCode 448) * **The Problem:** Multiple numbers can be missing. * **The Solution:** Use the **In-place Marking** technique. Iterate through the array; for each value x you see, go to index $\text{abs}(x)-1$ and multiply the value there by -1 . At the end, any index that still has a positive number was never “visited,” meaning that $\text{index}+1$ is a missing number. This satisfies the $O(1)$ space requirement.

190. Reverse Bits

To a Google L5/L6 engineer, “Reverse Bits” is a masterclass in **Bit Manipulation**. While a junior might try to convert the number to a string, reverse the string, and convert it back, a senior engineer knows that such an approach is slow and memory-intensive.

Instead, a senior engineer views the 32-bit integer as a **Stream of Bits** and uses low-level operators to “shift” and “mask” the data. This is how high-performance drivers and compression algorithms (like ZIP) actually work.

1. Problem Explanation

The Goal: You are given a 32-bit unsigned integer. You need to reverse the order of its bits.

What this means: If bit 0 (the rightmost bit) is a 1, it should move to position 31 (the leftmost bit). If bit 31 is a 0, it moves to position 0.

The “Why” this is tricky: * **Leading/Trailing Zeros:** A 32-bit integer always has 32 bits. If the number is 1, it is actually 0000...0001. Reversing it makes it a very large number starting with 1000... * **Integer Representation:** Computers store numbers in binary, but languages like JavaScript treat numbers as 64-bit floats under the hood, requiring careful use of “unsigned right shifts” to keep things 32-bit.

2. Solution Explanation

We use a technique called the **Shifting Window**. We iterate 32 times (once for each bit).

1. **Extract** the rightmost bit of the input.
2. **Shift** our result to the left to make room for this new bit.
3. **Place** the extracted bit into the result.
4. **Shift** the original input to the right to discard the bit we just processed.

Step-by-Step Visualization (Simplified for 4 bits) Let's imagine we are reversing 4 bits. Input $n = 3$, which is 0011 in binary.

```
=====
INITIAL STATE
=====
Input (n):  0 0 1 1 (Decimal 3)
Result (res): 0 0 0 0 (Starting at 0)

=====
ITERATION 1:
=====
1. Look at last bit of n:
   n:   0 0 1 [1]  <-- The '1'

2. Move res left to make room:
   res: 0 0 0 0 << 1 becomes 0 0 0 0

3. Put that '1' into res:
   res: 0 0 0 1

4. Discard last bit of n:
   n:   0 0 1 (n >> 1)

Result state: n = 001, res = 0001
=====
ITERATION 2:
=====
1. Look at last bit of n:
   n:   0 0 [1]

2. Move res left:
   res: 0 0 0 1 << 1 becomes 0 0 1 0

3. Put that '1' into res:
   res: 0 0 1 1

4. Discard bit of n:
   n:   0 0 (n >> 1)

Result state: n = 00, res = 0011
=====
ITERATION 3:
=====
1. Look at last bit of n: 0
```

2. Move res left:
res: 0 0 1 1 << 1 becomes 0 1 1 0

3. Put that '0' into res:
res: 0 1 1 0

4. Discard bit of n: n = 0

=====

ITERATION 4:

=====

1. Look at last bit of n: 0

2. Move res left:
res: 0 1 1 0 << 1 becomes 1 1 0 0

3. Put that '0' into res:
res: 1 1 0 0 (Decimal 12)

FINAL RESULT: 1100 (Bits are reversed from 0011)

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: Constant Time

The algorithm runs exactly 32 times.
32 is a fixed number that does not change regardless of
how large the input integer is.

Time = 32 * (Bitwise operations)
Time = Order of 32
In Big O, we call this Order of 1 (Constant Time).

SPACE COMPLEXITY DERIVATION: Constant Space

What are we storing?

1. The 'result' integer	---> 4 bytes
2. The loop counter	---> 4 bytes

The memory used does not scale with the input.
Total Space = Order of 1 (Constant Space).

4. Solution Code

Python Implementation

```
def reverseBits(n):
    res = 0
    # We must iterate exactly 32 times for a 32-bit integer
    for i in range(32):
        # 1. Extract the rightmost bit using the AND mask (& 1)
        # 2. Shift the result to the left to make space
        # 3. Add the extracted bit to the result using OR (/)
        bit = (n >> i) & 1
        res = res | (bit << (31 - i))

    # Return the final reversed number
    return res

# A more optimized 'Shift-As-You-Go' approach:
def reverseBits_optimized(n):
    res = 0
    for _ in range(32):
        # Shift result left to make room
        res <<= 1
        # Add the last bit of n to the new gap in res
        res |= (n & 1)
        # Shift n right to move to the next bit
        n >>= 1
    return res
```

JavaScript Implementation

```
/**
 * @param {number} n - a positive integer
 * @return {number} - a positive integer
 */
var reverseBits = function(n) {
    let res = 0;

    for (let i = 0; i < 32; i++) {
        // Step 1: Shift result left
        // Note: In JS, res << 1 treats res as a signed 32-bit int.
        res = res * 2;

        // Step 2: Get the last bit of n
        let bit = n & 1;

        // Step 3: Add bit to result
```

```

    res = res + bit;

    // Step 4: Shift n right
    // We use >>> (unsigned right shift) to handle large numbers correctly
    n = n >>> 1;
}

// Final result must be treated as unsigned
return res >>> 0;
};

```

Note 1: Terminology and Techniques

- **Masking (& 1):** A way to “mask” or hide all bits except the ones you care about. `n & 1` clears all bits except the very last one.
 - **Unsigned Right Shift (>>>):** In JavaScript, regular numbers are 64-bit. When you shift right, you want to make sure the “sign bit” isn’t copied over (which would fill the left with 1s for negative numbers). `>>>` fills the left with 0s always.
 - **Divide and Conquer (L6 Level Optimization):** For an L6, if this function is called billions of times, you can reverse bits in “chunks” using pre-computed tables (reversing 8 bits at a time using a 256-entry array). This reduces 32 iterations down to 4.
-

Note 2: Real-World Interview Variations

1. **Variation: Counting Set Bits (Hamming Weight - Google/Meta)** * **The Problem:** Count how many 1s are in a 32-bit integer. * **The Solution:** Similar loop. Use `n = n & (n - 1)`. This specific trick clears the rightmost set bit in every step, making the loop run only as many times as there are 1s, rather than a full 32 times.
2. **Variation: Power of Two (Bloomberg)** * **The Problem:** Check if a number is a power of 2. * **The Solution:** A power of two in binary looks like `1000...0`. If you subtract 1, it becomes `0111...1`. * **The Trick:** `return (n > 0) && (n & (n - 1) == 0)`. If a number is a power of 2, it shares no 1 bits with its predecessor.
3. **Variation: IP Address to Integer (Networking focus)** * **The Problem:** Convert an IP string like “192.168.1.1” to a 32-bit integer. * **The Solution:** Split by “.”, convert each chunk to a byte, and use bitwise shifts to pack them: `(byte1 << 24) | (byte2 << 16) | (byte3 << 8) | byte4`. This is exactly how routers process your internet traffic!

137. Single Number II

This problem is a classic test of an engineer's ability to manipulate data at the lowest possible level: **Bits**. A Google L5/L6 engineer recognizes that while a Hash Map solves this easily, the “Staff level” solution uses bitwise logic to achieve constant space, essentially building a **Modulo-3 Counter** using binary gates.

1. Problem Explanation

The Goal: You are given an array of integers where every element appears exactly **three times**, except for one element which appears exactly **once**. You need to find that single element.

The Constraints: To pass a top-tier interview, you must solve this in **Linear Time** and **Constant Space**. This means you cannot store the numbers in a Hash Map (which takes space proportional to the number of unique elements).

The Logic: If we look at the numbers in binary (1s and 0s), and we sum up the number of 1s at each specific bit position (the 1st bit, the 2nd bit, etc.), the total count for any bit position where the “tripled” numbers have a 1 will be a multiple of 3.

Example: [2, 2, 2, 5]

2 in binary: 0 1 0

2 in binary: 0 1 0

2 in binary: 0 1 0

5 in binary: 1 0 1

Sum of bits: 1 3 1

If we take each sum **Modulo 3** (the remainder after dividing by 3): $* 1 \% 3 = 1 * 3 \% 3 = 0 * 1 \% 3 = 1$ Result: 1 0 1, which is **5**.

2. Solution Explanation

We can implement this “Sum of bits” approach in two ways. An L5 engineer should know the “General” approach (looping through 32 bits) and the “Optimized” approach (using state machines).

Approach A: The 32-Bit Loop (Intuitive) We iterate from 0 to 31 (for 32-bit integers). At each position, we count how many numbers in the array have a 1 at that position.

Approach B: The Digital Logic Gate (Optimized) We use two variables, `ones` and `twos`, to act as a counter. * `ones`: Tracks bits that have appeared **1 time** (mod 3). * `twos`: Tracks bits that have appeared **2 times** (mod 3). * When a bit appears a **3rd time**, it should reset both `ones` and `twos` back to 0.

Step-by-Step ASCII Visualization of the State Machine: Imagine we are looking at just one bit position for the input [5, 5, 5]. (5 in binary is 101, let's focus on the rightmost bit, which is 1).

Initial State: `ones = 0, twos = 0`

```
=====
STEP 1: First '5' arrives (Bit is 1)
=====
```

Formula logic:

- Update 'ones': Should become 1 (since we've seen it once).
- Update 'twos': Remains 0.

Result: `ones = 1, twos = 0`

```
=====
STEP 2: Second '5' arrives (Bit is 1)
=====
```

Formula logic:

- Update 'ones': Should reset to 0 (we've seen it twice now).
- Update 'twos': Should become 1 (we've seen it twice).

Result: `ones = 0, twos = 1`

```
=====
STEP 3: Third '5' arrives (Bit is 1)
=====
```

Formula logic:

- Update 'ones': Would want to become 1, but...
- Update 'twos': Currently 1.
- SPECIAL MASK: If both 'ones' and 'twos' would be 1, reset both to 0.

Result: `ones = 0, twos = 0` (Back to base!)

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: Order of N

```
-----
Approach: 32-Bit Loop
```

- We loop 32 times (constant).

- Inside, we loop through the array of size N.
- Total operations: $32 * N$.
- Since 32 is a constant, this is $O(N)$.

Approach: State Machine (ones/twos)

- We loop through the array of size N exactly once.
- Inside, we do a few bitwise operations (XOR, AND, NOT).
- Total operations: $N * (\text{constant bitwise ops})$.

TIME COMPLEXITY VISUAL:

Input Elements (N) ----->

Processing time per element: | x | x | x | x |

Result: Order of N

SPACE COMPLEXITY DERIVATION: Order of 1

Approach: 32-Bit Loop

- We store: 1 result variable, 1 counter, 1 loop index.
- No matter how big N is, we only use these 3 variables.

Approach: State Machine

- We store: 2 variables (ones, twos).
- Space used does not grow with N.

SPACE COMPLEXITY VISUAL:

Input Size: [N]

Memory Used: [1] <-- Constant size

Result: Order of 1

4. Solution Code

Solution 1: The 32-Bit Loop (Very clear, great for interviews)

```
def singleNumber_loop(nums):
    result = 0
    # Iterate through every bit position (0 to 31)
    for i in range(32):
        count = 0
        bit_mask = 1 << i

        # Count how many numbers have a '1' at this specific bit
        for num in nums:
            if num & bit_mask:
                count += 1
```

```

        # If count is not divisible by 3, the single number has a '1' here
        if count % 3 != 0:
            # Handle Python's infinite precision integers for negative numbers
            if i == 31:
                result -= (1 << 31)
            else:
                result |= bit_mask

    return result

```

Solution 2: The State Machine (Optimized/Staff Level)

```

def singleNumber_optimized(nums):
    # 'ones' stores bits that appeared 1 time (mod 3)
    # 'twos' stores bits that appeared 2 times (mod 3)
    ones = 0
    twos = 0

    for num in nums:
        # 1. Update 'ones': Add the new num bits to 'ones'
        # but only if they aren't already in 'twos'.
        ones = (ones ^ num) & ~twos

        # 2. Update 'twos': Add bits that were already in 'ones'
        # and are appearing again, but only if they aren't in the new 'ones'.
        twos = (twos ^ num) & ~ones

    return ones

/**
 * @param {number[]} nums
 * @return {number}
 */
function singleNumber(nums) {
    let ones = 0;
    let twos = 0;

    for (let num of nums) {
        // XOR (^) adds the bit
        // AND NOT (& ~) ensures we don't count it if it belongs in the other state

        // If a bit is in 'twos', it shouldn't be in 'ones'
        ones = (ones ^ num) & ~twos;

        // If a bit is now in 'ones', it shouldn't be in 'twos'

```

```

        twos = (twos ^ num) & ~ones;
    }

    return ones;
}

```

Note 1: Terminology and Techniques

- **Bitwise XOR (^):** This is a “binary addition without carry.” $1 \oplus 1 = 0$. In the “Single Number I” problem (where numbers appear twice), XORing everything cancels out the pairs, leaving the single number.
 - **State Machine:** A design pattern where an object changes its behavior based on its current state. Here, each bit in our `ones` and `twos` variables acts as a state (00 -> 01 -> 10 -> 00) tracking the count of that bit.
 - **Two’s Complement:** How computers represent negative numbers. When working with bits in Python, the 31st bit represents the sign. A Senior engineer must handle this so the code works for negative inputs.
-

Note 2: Real-World / Interview Variations

1. Single Number III (Google / Bloomberg) * The Twist: Every number appears twice, but TWO numbers appear only once. Find both. * **L5 Solution:** XOR all numbers. The result is $A \oplus B$. Find any bit where A and B differ (the first 1 in the result). Use that bit to split the original array into two groups. One group will contain A, the other B. XORing each group individually reveals the two numbers.

2. Finding the “Majority Element” in a stream (Meta) * The Context: You are processing millions of IDs and need to find the one that appears more than 50% of the time with minimal memory. * **L5 Solution: Boyer-Moore Voting Algorithm.** It’s similar to our state machine logic. You keep a `candidate` and a `count`. If the next ID is the same as the candidate, `count++`. If different, `count--`. If `count` hits 0, the current ID becomes the new candidate.

3. Error Correction Codes (Google / Systems) * The Context: When data is sent over a wire, bits can flip due to interference. How do you detect which bit flipped? * **L5 Solution: Hamming Distance / Parity Bits.** This is the real-world application of this Leetcode problem. You store extra “parity” bits that represent the XOR sum of specific positions. If a bit flips, the parity sums will fail to match, allowing the system to pinpoint and flip the bit back to its correct state.

201. Bitwise AND of Numbers Range

Solving “Bitwise AND of Numbers Range” at a Google L5/L6 level requires moving past the “loop and calculate” mentality and looking for the mathematical invariant. This is a problem about **Binary Common Prefixes**.

1. Problem Explanation

The Goal: Given two integers `left` and `right`, return the bitwise AND of all numbers in that range, inclusive.

The “Brute Force” Trap: If `left = 1` and `right = 2,147,483,647`, a loop will run billions of times, resulting in a Time Limit Exceeded (TLE) error. We need a solution that works in constant time (or logarithmic time relative to the value of the numbers).

The Bitwise “Why”: In a bitwise AND operation, a bit in the result is 1 ONLY if that bit is 1 in every single number in the range. If a bit flips from 1 to 0 even once anywhere in the range, that bit in the final result becomes 0 forever.

2. Solution Explanation

The core insight is this: **The bitwise AND of a range is simply the common prefix of the binary representations of `left` and `right`.**

Everything to the right of the common prefix will eventually flip during the incrementing process, causing those bits to become 0.

Visualization of the “Flip” Let’s look at the range [9, 12]:

```
9:  1 0 0 1
10: 1 0 1 0
11: 1 0 1 1
12: 1 1 0 0
-----
AND:1 0 0 0
```

Notice that the first two bits 1 0 are common at the start, but as we count up, the second bit flips from 0 to 1 at 12. Wait, look closer: * Bit 3 (Value 8): Stays 1 for the whole range. * Bit 2 (Value 4): Is 0 in 9, 10, 11, then becomes 1 in 12. Since it was 0 at least once, the result bit is 0. * Bit 1 (Value 2): Flips multiple times. Result bit is 0. * Bit 0 (Value 1): Flips multiple times. Result bit is 0.

The common prefix of 9 (1001) and 12 (1100) is just the first bit “1”. Everything else gets zeroed out.

The L5/L6 Algorithm: “The Shift” We shift both `left` and `right` to the right until they become the same number. We count how many shifts we did. This “same number” is the common prefix. Then, we shift that number back to the left by the same count to fill the “flipped” positions with zeros.

Step-by-Step Walkthrough `left = 9, right = 12`

```
=====
INITIAL STATE
=====
left  (9):  1 0 0 1
right (12):  1 1 0 0
Shift Count: 0

=====
STEP 1: Shift Right
=====
9 >> 1  = 4  (Binary: 1 0 0)
12 >> 1 = 6  (Binary: 1 1 0)
4 != 6.
Shift Count: 1

=====
STEP 2: Shift Right
=====
4 >> 1 = 2  (Binary: 1 0)
6 >> 1 = 3  (Binary: 1 1)
2 != 3.
Shift Count: 2

=====
STEP 3: Shift Right
=====
2 >> 1 = 1  (Binary: 1)
3 >> 1 = 1  (Binary: 1)
1 == 1. MATCH FOUND!
Shift Count: 3

=====
STEP 4: Shift back to original magnitude
=====
Common Prefix: 1
Shift back left by 3: 1 << 3
Binary: 1 0 0 0  (Decimal: 8)

FINAL RESULT: 8
```

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(\log N)$

The number of bits in an integer is proportional to $\log(N)$.
For a 32-bit integer, the loop runs at most 32 times.

Work per iteration:

- Right shift: Order of 1
- Comparison: Order of 1
- Increment: Order of 1

Total Time = $32 * \text{Order of } 1 = \text{Order of } 1$ (strictly speaking)
Or more generally: Order of $\log(\text{Right_Value})$

SPACE COMPLEXITY DERIVATION: $O(1)$

We only store:

- shift_count (Integer)

Memory used does not depend on the size of the input range.
Total Space = Order of 1

4. Solution Code

Python Implementation

```
def rangeBitwiseAnd(left, right):
    # This is the 'Shift' approach (Common Prefix)
    shift_count = 0

    # Continue shifting until the numbers are identical.
    # This effectively removes the bits that change within the range.
    while left < right:
        left >>= 1
        right >>= 1
        shift_count += 1

    # Shift the common prefix back to the left to restore the number's magnitude
    return left << shift_count

# ALTERNATIVE: Brian Kernighan's Algorithm
```

```

# This is even faster as it jumps directly through the 1-bits.
def rangeBitwiseAnd_Optimized(left, right):
    # While right is greater than left, we clear the least significant
    # set bit of 'right' using: n & (n - 1)
    while left < right:
        right = right & (right - 1)
    return right

```

JavaScript Implementation

```

/**
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
var rangeBitwiseAnd = function(left, right) {
    let shiftCount = 0;

    // Find the common prefix by shifting right
    while (left < right) {
        left >>= 1;
        right >>= 1;
        shiftCount++;
    }

    // Shift back left to append zeros to the right of the common prefix
    // JavaScript uses 32-bit signed integers for bitwise operations
    return left << shiftCount;
};

```

Note 1: Terminology and Techniques

- **Common Prefix:** In binary terms, this is the set of high-order bits that two numbers share before they start to differ.
- **Brian Kernighan's Algorithm:** A technique to turn off the rightmost set bit of a number. The expression `n & (n - 1)` effectively changes the rightmost 1 to a 0. This is used here to quickly zero out the bits that would change in a range.

Note 2: Real World / Interview Variations

1. Variation: Number of Set Bits (Hamming Weight) (Meta/Bloomberg)

* **The Twist:** Count how many 1s are in a range. * **How to solve:** You can't use the prefix trick. You must use a mathematical approach (counting how

often each bit position toggles 1 across the range) or Brian Kernighan's on individual numbers.

2. Variation: Bitwise OR of Numbers Range (Google) * The Twist: Find the bitwise OR instead of AND. * **How to solve:** For OR, if a bit is 1 anywhere in the range, it becomes 1 in the result. The result will be the same common prefix as the AND solution, but instead of appending 0s, you append 1s.

3. Variation: IP Address Subnetting (Systems Design Context) * The Twist: You have a range of IP addresses. Find the smallest CIDR block (subnet mask) that contains them. * **How to solve:** This is EXACTLY the "Group Anagrams" of networking. An IP address is just a 32-bit integer. Finding the common binary prefix of the first and last IP in the range tells you exactly what the subnet mask (the "prefix") should be.

78. Subsets

This is an end-to-end, L5/L6 level walkthrough for the "Subsets" problem.

At a top-tier company, a Senior Engineer views this problem as a foundation for **Combinatorial Search**. They don't just see it as a list of numbers; they see it as a **Decision Tree**. For every single element in the input, you have a binary choice: "Do I include this element in my current set, or do I leave it out?"

1. Problem Explanation

The Goal: Given an integer array `nums` of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets.

What is a Subset? A subset is any combination of elements from the original array, including the empty set `[]` and the array itself `[1, 2, 3]`.

The Intuition: Imagine you are packing a bag from a pile of items. For the first item, you can either put it in the bag or leave it on the floor. Then you move to the second item and make the same choice. By the time you've looked at every item, the contents of your bag represent one unique subset.

2. Solution Explanation

We use a technique called **Backtracking**. It is a refined version of "Brute Force" where we explore a path, record the result, and then "backtrack" (undo our last step) to try a different path.

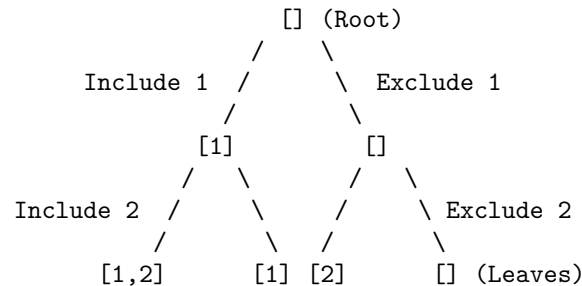
The Decision Tree Model Let's trace the input `nums = [1, 2]`.

=====

BACKTRACKING VISUALIZATION (Decision Tree)

=====

Start: Empty Set []



Final Subsets: [[1,2], [1], [2], []]

Step-by-Step Execution Walkthrough (`nums = [1, 2, 3]`) We use a recursive function. At each step, we decide whether to add `nums[i]` to our “current” list.

1. START: Current = [], Index = 0 (Target: 1)
 - CHOICE: Add 1.
 - Current is now [1]. Move to Index 1 (Target: 2).
2. STEP 2: Current = [1], Index = 1 (Target: 2)
 - CHOICE: Add 2.
 - Current is now [1, 2]. Move to Index 2 (Target: 3).
3. STEP 3: Current = [1, 2], Index = 2 (Target: 3)
 - CHOICE: Add 3.
 - Current is now [1, 2, 3]. Move to Index 3 (End).
 - SAVE [1, 2, 3].
 - BACKTRACK: Remove 3. Current is [1, 2].
4. STEP 4: Current = [1, 2], Index = 2 (Target: 3)
 - CHOICE: Skip 3.
 - Move to Index 3 (End).
 - SAVE [1, 2].
 - BACKTRACK: Remove 2. Current is [1].

... and so on.

3. Time and Space Complexity Analysis

Let N be the number of elements in the input array.

TIME COMPLEXITY DERIVATION: $O(N * 2^N)$

1. How many subsets are there?

For each of the N elements, we have 2 choices (In or Out).

Total subsets = $2 * 2 * 2 \dots$ (N times) = 2^N .

2. How much work per subset?

When we reach a "leaf" of our tree, we have to copy the current subset into our final result list.

On average, a subset has $N/2$ elements.

Copying takes $O(N)$ time.

Total Time = (Number of Subsets) * (Work to copy each)

Total Time = $2^N * N$

Overall Time Complexity = $O(N * 2^N)$

SPACE COMPLEXITY DERIVATION: $O(N)$

We don't count the space of the final output (which is $N * 2^N$).

We look at "Auxiliary Space" used during the process.

1. Recursion Stack:

The depth of our tree is exactly N (one level for each number).

Maximum stack depth = $O(N)$.

2. Current Path:

We store the current subset we are building.

Maximum size of this list = N .

Total Space = $O(N) + O(N)$

Overall Space Complexity = $O(N)$

4. Solution Code

Python Solution (Backtracking)

```
def subsets(nums):  
    result = []  
  
    # helper is a recursive function that builds the subsets  
    # curr: the subset we are building right now
```

```

# i: the index of the number we are currently considering
def backtrack(i, curr):
    # Base Case: If we have considered all numbers,
    # add a COPY of curr to our results.
    if i >= len(nums):
        result.append(list(curr)) # list() creates a new copy
        return

    # Decision 1: INCLUDE nums[i]
    curr.append(nums[i])
    backtrack(i + 1, curr)

    # Decision 2: EXCLUDE nums[i]
    # First, we must "undo" the append (Backtrack)
    curr.pop()
    backtrack(i + 1, curr)

backtrack(0, [])
return result

# Alternative Iterative Solution (often preferred for simplicity)
def subsets_iterative(nums):
    res = [[]]
    for n in nums:
        # For every existing subset, create a new one that includes n
        res += [curr + [n] for curr in res]
    return res

```

JavaScript Solution (Backtracking)

```

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var subsets = function(nums) {
    const result = [];

    function backtrack(index, current) {
        // Base Case: If pointer is at the end, record the subset
        if (index === nums.length) {
            // Use spread operator [...] to create a shallow copy
            result.push([...current]);
            return;
        }

        // Choice 1: Include the current element
    }

```

```

        current.push(nums[index]);
        backtrack(index + 1, current);

        // Choice 2: Exclude the current element (Backtrack)
        current.pop();
        backtrack(index + 1, current);
    }

    backtrack(0, []);
    return result;
};

// Iterative Solution
var subsetsIterative = function(nums) {
    let res = [[]];
    for (let n of nums) {
        let size = res.length;
        for (let i = 0; i < size; i++) {
            // Take every subset seen so far and add 'n' to it
            res.push([...res[i], n]);
        }
    }
    return res;
};

```

Note 1: Terminology and Techniques

- **Backtracking:** A strategy for finding all solutions by exploring a “state space” and abandoning a path as soon as it’s determined to be invalid (or finished). It relies on the “State Reset” (the `.pop()` in our code) to keep memory usage low.
 - **Power Set:** The mathematical term for the set of all subsets of a set, including the empty set and the set itself.
-

Note 2: Real World / Interview Variations

1. **Variation: Subsets II (Google/Meta) * The Twist:** The input array might contain duplicates (e.g., `[1, 2, 2]`). Your output must not have duplicate subsets. * **How to solve:** Sort the array first. During the backtracking, if you decide to *skip* a number, you must skip *all* subsequent instances of that same number to avoid generating the same subset twice.
2. **Variation: Permutations (Bloomberg) * The Twist:** Find all ways to arrange the numbers (e.g., `[1,2]`, `[2,1]`). * **How to solve:** This is also back-

tracking, but instead of an “In/Out” choice at each index, you iterate through *all* available numbers and pick one that hasn’t been used yet. You use a **used** boolean array to track choices.

3. Variation: File System Permissions (Systems Design/L6 context)

* **The Twist:** You have a set of user roles (Admin, Editor, Viewer). A user can have any combination of these. Calculate all possible “Access Profiles.” * **How to solve:** This is literally the Subsets problem. Each role is an element in the array. This is used in security auditing to ensure that unexpected combinations of permissions (subsets) don’t create security vulnerabilities.

231. Power of Two

A Google L5 (Senior) or L6 (Staff) engineer would view “Power of Two” not just as a math problem, but as a **Bit Manipulation** problem. While a junior might use a loop or recursion, a senior engineer looks for the most efficient way to communicate with the hardware.

In a high-level interview, an L6 would solve this in “Constant Time” using a single CPU operation.

1. Problem Explanation

The Core Goal: Given an integer *n*, return **true** if it is a power of two. Otherwise, return **false**. An integer *n* is a power of two if there exists an integer *x* such that $n = 2^x$.

The Patterns of Two: Let’s look at the numbers: $2^0 = 1$ $2^1 = 2$ $2^2 = 4$ $2^3 = 8$ $2^4 = 16$

The Binary Secret: Computers don’t see “8” or “16”; they see bits. If you look at powers of two in binary, a beautiful pattern emerges:

Decimal | Binary (8-bit)

1	00000001
2	00000010
4	00000100
8	00001000
16	00010000
32	00100000

Observation: Every power of two has **exactly one** bit set to 1. Every other number has either zero bits set (the number 0) or multiple bits set (like 3, which is 0011, or 6, which is 0110).

2. Solution Explanation

To solve this efficiently, we use a famous bitwise trick: $n \& (n - 1)$.

The Logic behind $n \& (n - 1)$ When you subtract 1 from a number, the rightmost 1 bit turns into a 0, and all the 0s to its right turn into 1s.

- **Case A: If n is a power of two (Only one 1 bit)** If we perform a bitwise AND between n and $n - 1$, the only 1 bit will cancel out, resulting in exactly 0.
- **Case B: If n is NOT a power of two (Multiple 1 bits)** The AND operation will only flip the rightmost 1, leaving the other 1 bits intact. The result will NOT be 0.

Step-by-Step Visualizations Example 1: $n = 8$ (Is a power of two)

$n = 8$ $\rightarrow$ Binary: 1 0 0 0
 $n - 1 = 7$ $\rightarrow$ Binary: 0 1 1 1

Perform Bitwise AND (&):

```
  1 0 0 0 (8)
& 0 1 1 1 (7)
-----
  0 0 0 0 (Result is 0!)
```

Conclusion: 8 is a power of two.

Example 2: $n = 6$ (Is NOT a power of two)

$n = 6$ $\rightarrow$ Binary: 0 1 1 0
 $n - 1 = 5$ $\rightarrow$ Binary: 0 1 0 1

Perform Bitwise AND (&):

```
  0 1 1 0 (6)
& 0 1 0 1 (5)
-----
  0 1 0 0 (Result is 4, NOT 0!)
```

Conclusion: 6 is NOT a power of two.

Non-Trivial Edge Case: $n = 0$ or Negative Numbers A power of two must be a positive integer. $*0 \& -1$ is 0, but 0 is not a power of two. * Negative numbers are not powers of two. Therefore, our code must first check if $n > 0$.

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(1)$

The operation involves:

1. One comparison ($n > 0$)
2. One subtraction ($n - 1$)
3. One bitwise AND ($\&$)
4. One comparison to zero ($== 0$)

These are all primitive CPU instructions that take a fixed amount of time regardless of how large 'n' is.

Total Time = Constant Time = $O(1)$

SPACE COMPLEXITY DERIVATION: $O(1)$

We do not use any:

- Arrays
- Hash Maps
- Recursion (no stack space)

We only use the space already occupied by the input variable itself.

Total Space = Constant Space = $O(1)$

4. Solution Code

Optimized Solution (The Bitwise Trick) This is the L5/L6 preferred solution because it is the fastest possible way for a computer to calculate this.

Python

```
def isPowerOfTwo(n):  
    # Rule 1: n must be positive (powers of two are 1, 2, 4...)  
    # Rule 2: n AND (n-1) must result in 0  
    return n > 0 and (n & (n - 1)) == 0
```

JavaScript

```
function isPowerOfTwo(n) {  
    /**  
     * In JavaScript, bitwise operators convert the number  
     * to a 32-bit integer.  
     * n & (n - 1) removes the rightmost set bit.
```

```

        */
    return n > 0 && (n & (n - 1)) == 0;
}

```

Alternative Solution (The Loop/Iterative Approach) Useful to mention in an interview to show you understand the mathematical definition, though less efficient than the bitwise version.

Python

```

def isPowerOfTwo_iterative(n):
    if n <= 0:
        return False
    # Keep dividing by 2 as long as the number is even
    while n % 2 == 0:
        n = n // 2
    # If it was a power of two, we must end up at 1
    return n == 1

```

Note 1: Terms and Techniques

- **Bitwise AND (&):** A binary operator that compares each bit of two numbers. It returns 1 only if both bits are 1.
 - **The $n \& (n-1)$ Trick:** A powerful bit manipulation technique used to clear the least significant set bit. It's used in algorithms like "Counting Set Bits" (Hamming Weight).
 - **Constant Time $O(1)$:** An algorithm that always takes the same amount of time to execute, regardless of input size. In the software industry, $O(1)$ is the gold standard for performance.
-

Note 2: Real-World / Interview Variations

1. Variation: "Counting Bits" (Meta) * The Problem: Given an integer n , return how many 1s are in its binary representation. * **How to solve:** You can use a loop and the $n \& (n-1)$ trick. Every time you perform $n = n \& (n-1)$, one 1 bit disappears. You count how many times you can do this before the number becomes 0.

2. Variation: "Power of Four" (Google) * The Problem: Check if n is a power of 4 (1, 4, 16, 64). * **How to solve:** A power of 4 is *also* a power of 2. So first, check if it's a power of 2 using $n \& (n-1)$. Second, a power of 4 always has the 1 bit in an odd position (1st, 3rd, 5th...). You check this using a bitmask: $(n \& 0x55555555) != 0$.

3. Variation: “Unique Paths / Resource Allocation” (Bloomberg) *

The Problem: In some low-level systems, memory blocks must be “Power of Two” aligned (e.g., 1024 bytes, 2048 bytes). * **How to solve:** When writing a memory allocator, you use the `isPowerOfTwo` check to validate that a user’s request for memory alignment is valid for the hardware’s architecture. If a user asks to align memory by 7 bytes, the system throws an error because 7 is not a power of 2.

318. Maximum Product of Word Lengths

Solving “Maximum Product of Word Lengths” at a Google L5/L6 level requires moving beyond simple string comparisons to **Bit Manipulation**. A top-tier engineer looks for ways to transform expensive operations (like checking if two strings share characters) into cheap operations (like a single CPU instruction).

1. Problem Explanation

The Goal: You are given an array of strings called `words`. You need to find the maximum value of `length(words[i]) * length(words[j])`, but there is a strict condition: * The two words, `words[i]` and `words[j]`, must **not share any common letters**.

The Naive Approach (The Trap): A junior approach would be to compare every pair of words. For each pair, you would loop through the characters of one word and check if they exist in the second word.

*If you have 1000 words, and each word is 100 letters long, you are doing 1000 * 1000 * 100 comparisons. This is extremely slow.**

The “Why” of the Engineering Solution: We need a way to represent the “character set” of a word so that we can compare two words instantly. Since there are only 26 lowercase English letters, we can use a **Bitmask**.

2. Solution Explanation

The Bitmask Concept Imagine a row of 26 light switches, one for each letter from ‘a’ to ‘z’. * If a word contains ‘a’, we flip the 0th switch. * If a word contains ‘b’, we flip the 1st switch. * If a word contains ‘z’, we flip the 25th switch.

This row of switches is just an integer (a 32-bit integer has plenty of room for 26 bits).

The “Magic” Comparison If we have two bitmasks, `mask1` and `mask2`: * We perform a **Bitwise AND** (`mask1 & mask2`). * If the result is **0**, it means no switches were “ON” in both masks. Therefore, the words share no letters! * If the result is **NOT 0**, they share at least one letter.

ASCII Visualization of Bitmasking:

```
Word A: "abc"
Letters: a, b, c
Bitmask (Binary): 0000...000000000000000000111
                    |||
                    cba
```

```
Word B: "def"
Letters: d, e, f
Bitmask (Binary): 0000...0000000000000000111000
                    |||
                    fed
```

```
-----
CHECKING FOR COMMON LETTERS: (A & B)
  000...000111 (Word A)
& 000...111000 (Word B)
-----
  000...000000 (Result is 0 -> NO COMMON LETTERS!)
-----
```

```
Word C: "cat"
Letters: c, a, t
Bitmask: Contains bit for 'c' and 'a'.
```

```
CHECKING A & C:
  000...000111 (Word A: "abc")
& 000...100101 (Word C: "cat")
-----
  000...000101 (Result is NOT 0 -> THEY SHARE 'a' and 'c'!)
```

The L5/L6 Optimization: Bitmask Map If multiple words have the same characters (e.g., “cool” and “loco”), they will produce the exact same bitmask. We only care about the **longest** word for any given bitmask. We can use a Hash Map to store `mask -> max_length`.

Step-by-Step Walkthrough: Input: `["abcw", "baz", "foo", "bar", "xtfn", "abcdef"]`

```
=====
STEP 1: PRE-PROCESSING (Build Mask Map)
```

```

=====
1. "abcw"    -> Mask: (bits 0,1,2,22) -> Map: {Mask_abcw: 4}
2. "baz"     -> Mask: (bits 0,1,25)   -> Map: {..., Mask_baz: 3}
3. "foo"     -> Mask: (bits 5,14)     -> Map: {..., Mask_foo: 3}
... and so on.

=====

STEP 2: NESTED COMPARISON (Compare Masks)
=====

We loop through the keys in our Map (Unique Masks).

Pair 1: "abcw" (MaskA) and "xtfn" (MaskB)
MaskA & MaskB == 0? YES.
Product: 4 * 4 = 16. Update MaxProduct = 16.

Pair 2: "abcw" (MaskA) and "abcdef" (MaskC)
MaskA & MaskC == 0? NO (Result is > 0).
Skip.

=====

FINAL RESULT: 16
=====

```

3. Time and Space Complexity Analysis

Let N be the number of words and L be the average length of a word.

TIME COMPLEXITY DERIVATION: $O(N * L + N * N)$

Part 1: Generating Masks

We loop through every word (N) and every character (L).

Time = $N * L$

Part 2: Finding Max Product

In the worst case (all unique masks), we compare every pair of masks.

Time = $N * N$

Total Time = $(N * L) + (N * N)$

SPACE COMPLEXITY DERIVATION: $O(N)$

We store one integer mask and one integer length for each word in a Hash Map.

```
[ Mask 1 : Len ]
[ Mask 2 : Len ]
...
[ Mask N : Len ]
```

Total Space = $O(N)$

4. Solution Code

Python Implementation

```
def maxProduct(words):
    # Map to store the bitmask of a word as key and its max length as value
    # mask_map = { bitmask: max_length }
    mask_map = {}

    for word in words:
        mask = 0
        for char in word:
            # Shift 1 left by (ASCII of char - ASCII of 'a')
            # 'a' becomes 1, 'b' becomes 2 (binary 10), 'c' becomes 4 (binary 100)
            mask |= (1 << (ord(char) - ord('a')))

        # If we've seen this mask before, only keep the longest length
        mask_map[mask] = max(mask_map.get(mask, 0), len(word))

    max_prod = 0

    # Compare every unique mask against every other unique mask
    masks = list(mask_map.keys())
    for i in range(len(masks)):
        for j in range(i + 1, len(masks)):
            # BITWISE AND check: if result is 0, no common letters
            if (masks[i] & masks[j]) == 0:
                prod = mask_map[masks[i]] * mask_map[masks[j]]
                max_prod = max(max_prod, prod)

    return max_prod
```

JavaScript Implementation

```
/**
 * @param {string[]} words
 * @return {number}
```

```

*/
var maxProduct = function(words) {
  const maskMap = new Map();

  for (const word of words) {
    let mask = 0;
    for (let i = 0; i < word.length; i++) {
      // Compute bitmask: 1 << (char code - 97)
      mask |= (1 << (word.charCodeAt(i) - 97));
    }

    // Store only the longest length for each unique character set
    const currentMax = maskMap.get(mask) || 0;
    if (word.length > currentMax) {
      maskMap.set(mask, word.length);
    }
  }

  let maxProd = 0;
  const masks = Array.from(maskMap.keys());

  // Nested loop to compare masks
  for (let i = 0; i < masks.length; i++) {
    for (let j = i + 1; j < masks.length; j++) {
      // Logical AND of bitmasks
      if ((masks[i] & masks[j]) === 0) {
        const prod = maskMap.get(masks[i]) * maskMap.get(masks[j]);
        maxProd = Math.max(maxProd, prod);
      }
    }
  }

  return maxProd;
};

```

Note 1: Terms & Techniques

- **Bitmask:** Using the bits of an integer to represent a set of boolean values (like “Does this word have the letter ‘A’?”).
- **Bitwise AND (&):** A comparison that returns 1 only if the bits in both numbers are 1. In this problem, it acts as an “Intersection” operation for sets.
- **Left Shift (<<):** Moving the bits of a number to the left. $1 \ll 2$ turns 0001 into 0100. This is used to target a specific “slot” in our bitmask for

each letter.

Note 2: Real-World Interview Variations

1. Permission Systems (Google / Cloud Infrastructure) * Problem:

You have users and a set of 30 different permissions (Read, Write, Execute, Admin, etc.). Check if User A and User B have overlapping permissions. *

Solution: This is exactly the same as the word problem. You assign each permission a bit (0 to 29). User permissions are stored as a single integer bitmask. Checking for overlap is a single `maskA & maskB > 0` operation.

2. Duplicate File Detection (Meta) * Problem:

Find files that have no common tags or metadata. * **Solution:** If the number of possible tags is small (under 64), you use a bitmask for the tags. This allows you to filter out trillions of pairs of files instantly using bitwise logic before doing expensive content comparisons.

3. Efficient Sudoku Solver (Bloomberg) * Problem:

Determine if a number can be placed in a Sudoku cell without checking the entire row/column/box every time. * **Solution:** Use three arrays of bitmasks (one for rows, one for columns, one for 3x3 boxes). A bitmask `row_masks[i]` stores which numbers are already used in row `i`. To see if number 5 is available, check `(row_masks[i] & (1 << 5)) == 0`. It turns a linear search into a constant time check.